

A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development

Jianguo Chen, Zeju Cai, Wenqing Chen, Weicheng Wang, Zibin Zheng, *Fellow, IEEE*,
and Philip S. Yu, *Life Fellow, IEEE*

Abstract—Large Language Models (LLMs) have achieved remarkable progress in code intelligence tasks, significantly enhancing the efficiency of software development. However, several challenges remain. First, fine-tuning LLMs for specific tasks requires a large amount of task-specific labeled data, which is often costly and time-consuming to acquire. Second, due to the sensitivity of code data, high-quality internal datasets from different organizations cannot be directly shared or combined for fine-tuning. Moreover, variations in programming languages across organizations can introduce interference during the fine-tuning process. To address these challenges, we propose F-CodeLLM, a federated adaptive large language model fine-tuning framework designed for real-world software development scenarios. To the best of our knowledge, this is the first approach to apply federated learning to the fine-tuning of code LLMs, enabling collaborative model optimization while preserving the privacy of each organization's code data. We design an efficient LLM fine-tuning method to mitigate the computational and communication overhead associated with collaborative fine-tuning. Experimental results demonstrate that F-CodeLLM effectively allows LLMs to learn from each organization's dataset, achieving performance comparable to centralized fine-tuning. Furthermore, F-CodeLLM is well-suited for multilingual data environments, as it can leverage shared knowledge across programming languages to enhance performance within individual language domains. Our code is publicly available at: <https://github.com/AAony/F-CodeLLM>.

Index Terms—Code intelligence, federated fine-tuning, large language model, software development

I. INTRODUCTION

WITH the rapid advancement of Natural Language Processing (NLP), Large Language Models (LLMs) have emerged as a major milestone in the field. Compared with earlier language models such as GloVe [1], LLMs exhibit significantly stronger performance across a wide spectrum of NLP tasks, including machine translation and question answering [2], [3]. Recognizing that source code possesses structural and semantic characteristics analogous to natural language, researchers have extended LLMs to code-related applications, such as code generation, summarization, and translation [4]. The emergence of code-oriented LLMs has further accelerated the paradigm shift toward Software 3.0, an era in which AI models can autonomously assist even lead software engineering processes [5]. Despite the impressive zero-shot and few-shot generalization capabilities of general-purpose LLMs, their

performance often lags behind task-specific fine-tuned models [6]. For example, models such as CodeX tend to underperform on domain-specific programming languages like SQL, which contains multiple dialects that general LLMs are typically not trained to distinguish effectively [7]. In practice, fine-tuning on domain-relevant data remains crucial to maximize the utility of LLMs in specialized software development scenarios. The effectiveness of this process, however, hinges on access to high-quality, task-specific training data.

Public code repositories such as GitHub offer a wide range of programming examples; however, they often fail to reflect the internal conventions, documentation styles, and system architectures characteristic of proprietary or enterprise-level code bases [8]. In real-world software development, each organization typically maintains code with a unique structure, domain-specific APIs, and customized business logic. Therefore, to effectively adapt LLMs for downstream software engineering tasks, organizations need to fine-tune models on their private code data. However, this process is often infeasible under a centralized learning paradigm, primarily due to the sensitive nature of code assets, competitive confidentiality, and increasingly stringent privacy regulations such as GDPR and HIPAA [9], [10]. Moreover, the process of curating and labeling high-quality training data is both costly and time-intensive, further discouraging organizations from sharing their datasets openly.

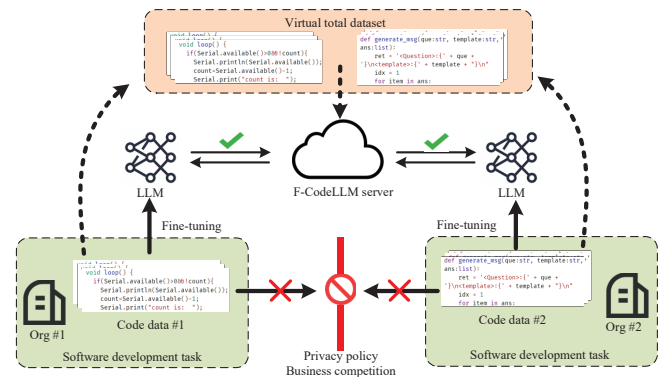


Fig. 1. Example of collaborative software development across multiple organizations using the F-CodeLLM framework. Organizations can collaboratively fine-tune the model while preserving the privacy of their local code data, ultimately producing a model that outperforms those fine-tuned individually.

To address these challenges, Federated Learning (FL) provides a promising alternative. FL enables collaborative model training across multiple organizations without the need to transfer local data, thereby preserving privacy and data

Jianguo Chen, Zeju Cai, Wenqing Chen, Weicheng Wang, and Zibin Zheng are with the School of Software Engineering and Zhuhai Key Laboratory of Trusted Large Language Models, Sun Yat-sen University, 519082 China (e-mails: {chenjg33, caizj9, chenwq95, wangwch7, zhizbin}@mail.sysu.edu.cn). Philip S. Yu is with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL 60607 USA (e-mail: psyu@uic.edu).

sovereignty [11]. Initially applied to domains such as keyboard prediction in Gboard, FL has since been successfully extended to NLP tasks [12]. In addition, FL offers benefits particularly relevant to software engineering, including personalization, robustness, diversity, and regulatory compliance [13]. However, applying FL to large code LLMs introduces new technical challenges. **Traditional FL requires frequent transmission of model parameters, which becomes prohibitively expensive for large-scale LLMs.** Moreover, full fine-tuning of LLMs demands substantial memory and computational resources, often exceeding the capacity of participating clients. These constraints present significant practical obstacles to the adoption of FL in real-world software organizations.

In this paper, we propose **F-CodeLLM, a federated adaptive large language model fine-tuning framework** for software development. F-CodeLLM updates models using small, trainable adapter modules instead of the full set of model parameters, thereby significantly reducing both communication overhead and local resource demands. An example of collaborative software development across multiple organizations using the F-CodeLLM framework is illustrated in Fig. 1. To the best of our knowledge, we are the first to apply FL to address the challenge of limited code data for LLM fine-tuning on specific downstream tasks. Our main contributions are as follows:

- (1) We employ FL to address the challenge of limited code data for LLMs during fine-tuning on specific downstream tasks. When the code languages vary across organizations, F-CodeLLM is **capable of effectively capturing the shared features of different code languages**, thereby enhancing performance on individual code languages.
- (2) **To mitigate the high communication overhead and computational burden associated with federated fine-tuning of LLMs, we propose an efficient joint fine-tuning method based on LoRA**, which substantially reduces both computational and network demands at the edge.
- (3) We evaluate F-CodeLLM on **three code-related downstream tasks**. Experimental results demonstrate that it effectively aggregates knowledge from multiple organizations, significantly improving model performance while reducing network traffic and computational resource consumption compared to traditional federated learning approaches.

The remainder of the paper is organized as follows. Section II provides a brief discussion to the related work in this field. Section III presents an overview of F-CodeLLM. Section V introduces the comparison experiments. Finally, Section VI concludes this work.

II. RELATED WORK

Extensive research has focused on applying NLP models to software engineering with the goal of enhancing the intelligence of software development tools, processes, and systems [14], [15]. For instance, Lin *et al.* proposed replacing MSP-RNN with a transformer to identify anomalies in logs, addressing the challenges of parallelization and capturing long-term dependencies associated with RNNs [14]. In [15], Fu *et al.* introduced a GPT-2-based model for estimating agile

story points, demonstrating the potential of NLP models to support project management tasks.

Pre-training techniques have developed rapidly. At the same time, the size of model parameters has been continuously increasing. Models based on the transformer architecture show certain abilities that only appear in larger models. Smaller models usually do not have these abilities. We refer to models possessing these capabilities as LLMs [16]. Code LLMs are designed to address code-related challenges. They are often built upon high-performing NLP models and are further optimized for code intelligence tasks. For example, CodeT5 is based on the T5 architecture that has been optimized for code intelligence tasks by modifying its training objectives [9]. Other models, such as **CodeGen [4] and AlphaCode [17]**, have also been developed to tackle code-related tasks. The pre-training corpora for code LLMs typically consist of code and text data collected from code hosting platforms such as GitHub and StackOverflow [18].

The massive number of parameters in LLMs requires substantial computational resources and time for full-parameter fine-tuning [19]. Recently, researchers have proposed Parameter-Efficient Fine-Tuning (PEFT) methods that significantly reduce the computational and memory demands of fine-tuning LLMs. For example, Hu *et al.* proposed **LoRA, a method that reduces the number of trainable parameters during fine-tuning, substantially lowering the hardware requirements for LLMs** [20]. Other approaches, such as P-Tuning [21], Prefix Tuning [22], and Prompt Tuning [23], also reduce hardware demands during fine-tuning. However, these methods primarily focus on centralized fine-tuning. Zhang *et al.* explored the combination of LoRA and federated learning for **general instruction fine-tuning** [24]. However, they did not fully address the specific challenges associated with code-related tasks.

Recent advances have combined FL with parameter-efficient tuning (PET) methods to address the challenges of training large models under privacy constraints [25], [26]. Notably, Che *et al.* proposed FedPrompt, which employs federated prompt tuning to adapt pre-trained language models using learned soft prompts that are optimized collaboratively across clients [25]. Zhang *et al.* designed FedPETuning, which integrates adapter modules and prefix tuning within an FL framework [27]. FedPETuning can reduce communication overhead and improve personalization in general NLP tasks. More recently, Babakniya *et al.* presented SLoRA, which applies LoRA-based fine-tuning in a federated context, **effectively reducing both trainable parameters and communication costs** [28].

While current approaches have demonstrated success in NLP domains, they have not been evaluated on code-specific downstream tasks involving multiple programming languages and heterogeneous client data. In contrast, our **F-CodeLLM is specifically designed for the fine-tuning of code LLMs, addressing challenges unique to code data, such as cross-language generalization and non-IID distributions across organizations**. Furthermore, F-CodeLLM leverages LoRA adapters not only to improve communication efficiency but also to facilitate the learning of shared representations, thereby enhancing performance on diverse code tasks in a collaborative manner.

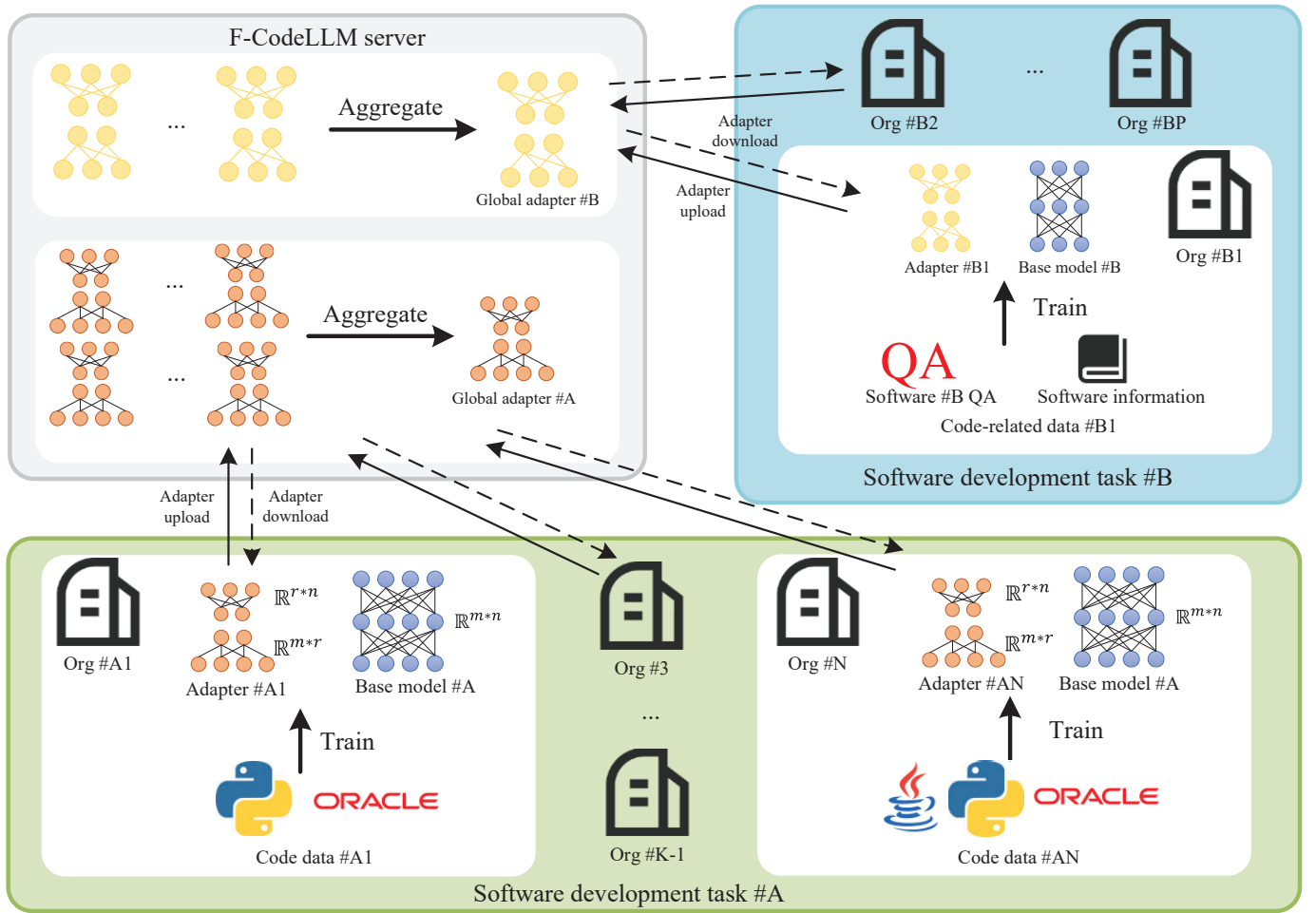


Fig. 2. The proposed F-CodeLLM framework. F-CodeLLM performs federated fine-tuning of LLMs across multiple distinct downstream software development tasks.

III. METHODOLOGY

A. Framework Overview

We propose F-CodeLLM, a novel federated learning framework designed to collaboratively fine-tune code LLMs across multiple organizations without compromising data privacy. In real-world software engineering scenarios, organizations often possess valuable, task-specific code datasets that cannot be shared due to privacy policies, regulatory constraints, or competitive considerations. This fragmented and siloed data landscape severely limits the fine-tuning and deployment of domain-adapted LLMs. F-CodeLLM is the first framework to address this challenge by enabling federated fine-tuning of code LLMs across organizations in a communication- and computation-efficient manner. The workflow of the proposed F-CodeLLM framework is illustrated in Fig. 2.

The F-CodeLLM architecture consists of two main components:

- **Organization Side:** Each participating organization downloads a shared base model and performs local fine-tuning on task-specific code data, updating only the LoRA adapter. The fine-tuning process is resource-adaptive, allowing each participant to select batch sizes and compute settings based on local infrastructure.

- **Server Side:** A trusted server collects adapter updates and sample counts from all participants. It then performs weighted aggregation of adapter parameters and redistributes the updated adapters for the next round of training.

Training continues for a pre-agreed number of communication rounds. After each round, the server evaluates convergence criteria and orchestrates the next iteration. Once training is complete, each organization retains a customized model enhanced by the shared knowledge learned from diverse code datasets all without exposing any raw code to external parties.

F-CodeLLM addresses three core technical challenges:

- **Challenge 1: Communication bottlenecks in FL with code LLMs.** Fine-tuning LLMs in a federated setting incurs high communication costs due to the large size of model parameters. To address this, F-CodeLLM employs a lightweight LoRA-based fine-tuning mechanism, which freezes the base model and updates only compact adapter modules. This mechanism significantly reduces the amount of data exchanged during each communication round, making the framework practical for bandwidth-constrained environments.
- **Challenge 2: Heterogeneous and Non-IID code data across organizations.** Organizations may work with

different programming languages or downstream tasks, resulting in heterogeneous and non-IID data distributions. F-CodeLLM shows that shared adapter training across diverse clients enables the model to learn cross-language, task-agnostic representations, thereby improving generalization and enhancing performance on individual client tasks.

- **Challenge 3: Efficient coordination of distributed fine-tuning.** Coordinating the fine-tuning process across distributed clients with varying computational resources poses both system-level and algorithmic challenges. F-CodeLLM defines a clear protocol for model initialization, local training, adapter uploading, and server-side weighted aggregation. This process is robust to client variability and scales effectively to real-world deployment scenarios.

B. Local Fine-Tuning

In the local fine-tuning process, prior to training, organizations must preprocess their datasets and convert them into code data suitable for model training through dedicated prompts and tokenizers. These processed datasets are then used to fine-tune the LLM. F-CodeLLM uses LoRA adapters, which inject low-rank matrices into each trainable matrix of the Transformer. These adapters perform the fine-tuning operations instead of updating the full model parameters, thereby reducing computational and memory requirements. The local training process of an F-CodeLLM organization is illustrated in Fig. 3.

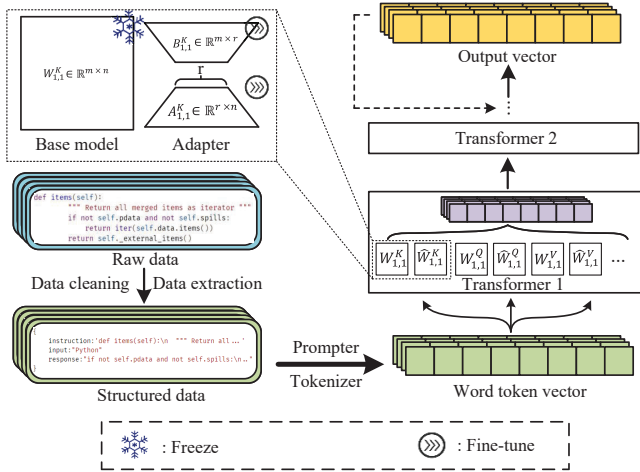


Fig. 3. Local training process in the F-CodeLLM framework. Each organization manages its own local data and performs federated fine-tuning using the F-CodeLLM algorithm. Prior to training, organizations must carefully preprocess their local datasets and convert them into code data suitable for model training using specific prompts and tokenizers. During fine-tuning, adapters inject low-rank matrices into each trainable matrix of the Transformer, allowing parameter-efficient updates without directly modifying the original model weights.

Adapters are incorporated into each Transformer module of the LLM, with trainable low-rank decomposition matrices injected into the trainable matrices of the self-attention module. The trainable matrices include the query matrix W^Q , key matrix W^K , value matrix W^V , and output matrix

W^O). Specifically, for the base model W , consider the j -th key matrix within its i -th Transformer module, denoted as $W_{i,j}^K \in \mathbb{R}^{m \times n}$. The adapter introduces the low-rank matrix $\hat{W}_{i,j}^K$ into this matrix, eliminating the need to directly fine-tune $W_{i,j}^K$. The matrix $\hat{W}_{i,j}^K$ comprises two submatrices, namely:

$$\begin{aligned} A_{i,j}^K &\in \mathbb{R}^{r \times n} \\ B_{i,j}^K &\in \mathbb{R}^{m \times r} \end{aligned} \quad (1)$$

where r denotes the rank of the matrix, which is significantly smaller than both m and n , and $\hat{W}_{i,j}^K = B_{i,j}^K \times A_{i,j}^K \in \mathbb{R}^{m \times n}$. The adapter $\hat{W}$ comprises all low-rank matrices injected into the base model. Both W and $\hat{W}$ operate on the same input x , and the resulting vectors are then summed element-wise. Assuming only the base model W is used, let $h = Wx$. The output when the adapter is used can be expressed as:

$$h' = Wv + \hat{W}v = (W + \hat{W})v = (W + \hat{B}\hat{A})v. \quad (2)$$

Due to the gradient instability encountered when training large models, fine-tuning with heterogeneous data across organizations can further exacerbate gradient divergence. To tackle this issue, we utilize an L2-norm constraint to constrain model divergence during updates. The optimization objective is defined as:

$$\min_{\hat{w}} \left(\sum_{k=1}^K \frac{1}{B} \sum_{i=1}^B \mathcal{L}(w + \hat{w}; x_{k,i}, y_{k,i}) + \lambda \|\hat{w} - \hat{w}^c\|_2^2 \right), \quad (3)$$

where $\mathcal{L}$ denotes the loss function, and $\hat{w}^c$ represents the globally aggregated adapter derived from the prior communication round. K denotes the number of data batches per organization, while B denotes the batch size. λ is the coefficient of the regularization term, which can be tuned based on practical considerations.

In the context of multi-programming language joint fine-tuning in F-CodeLLM, each organization incorporates an additional language identifier into the structured code data. This method enables LLMs to recognize and differentiate between code snippets written in various programming languages. It also allows the models to learn shared features and patterns across languages during the training process. Consequently, F-CodeLLM does not necessitate that all organizations employ the same programming language; instead, it emphasizes the presence of similar downstream tasks among participating entities.

C. Federated Fine-tuning

In federated learning, data remains within each organization, with only the trained adapters transmitted between organizations and the server. A key limitation of FL lies in its high demands on network bandwidth and computational resources. These demands escalate as the model size increases, with larger models imposing greater strain on both network infrastructure and local computing capabilities. This limitation becomes particularly pronounced when training extremely large models.

To address this challenge, we propose a LoRA-based federated fine-tuning algorithm, which reduces the training of large

models to the training of compact adapters, as described in the previous section. To demonstrate the communication overhead savings achieved by F-CodeLLM, we conduct simulations of the federated fine-tuning process using LLaMA-7B [29]. The results are presented in Fig. 4.

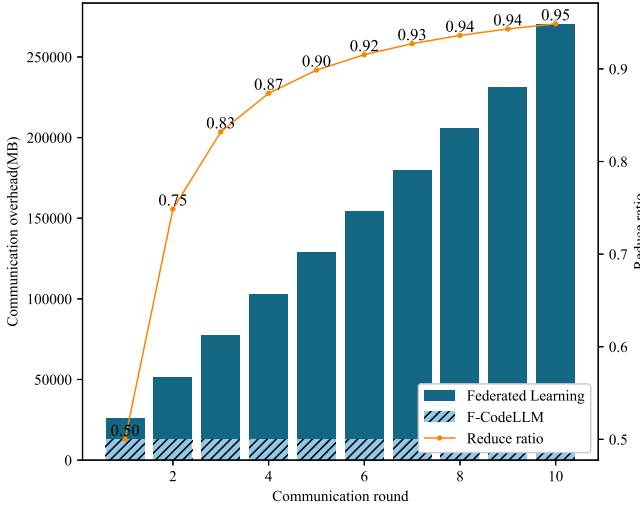


Fig. 4. Comparison of communication overhead between FL and F-CodeLLM. The bars in the figure denote the communication costs of two different fine-tuning methods for LLMs, while the line indicates the proportion of communication cost savings achieved by F-CodeLLM relative to FL.

We perform 10 communication rounds to calculate the network overhead for a single organization in F-CodeLLM. As evident from the results, our method significantly reduces communication overhead, with the reduction rate increasing alongside the number of communication rounds.

Assuming there are N organizations participating in the fine-tuning of the code LLM W , which are denoted as $K = \{K_1, K_2, \dots, K_N\}$. The aggregated code datasets from all organizations are represented as $\mathcal{Z} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, containing a total of n data samples, where x_i and y_i form a context-target pair. The dataset belonging to organization K_k is denoted as $\mathcal{Z}_k = \{(x_1, y_1), (x_2, y_2), \dots, (x_{n_k}, y_{n_k})\}$, with n_k samples, and $n = \sum_{k=1}^K n_k$. During the fine-tuning process, we freeze the base model W and introduce a compact adapter $\hat{W}$ rather than training W itself. The ultimate goal of F-CodeLLM is to optimize

$$\max_{\hat{w}} \sum_{(x,y) \in \mathcal{Z}} \sum_{u=1}^{|y|} \log(P_{w+\hat{w}}(y_u | x, y_{<u})), \quad (4)$$

where $\hat{w}$ denotes the global adapter, and $P_{w+\hat{w}}(y_u | x, y_{<u})$ represents the probability that the model outputs the u -th target given context x and targets of length less than u as input. However, due to the distributed nature of data across different organizations, it is infeasible to train on all the data in a single location. Consequently, the optimization objective of F-CodeLLM must be reformulated as:

$$\max_{\hat{w}} \sum_{k=1}^K \frac{n_k}{n} \sum_{(x,y) \in \mathcal{Z}_k} \sum_{t=1}^{|y|} \log(P_{w+\hat{w}}(y_t | x, y_{<t})). \quad (5)$$

On the server side, we assign distinct aggregation weights to the adapters of each organization in accordance with the FedAvg algorithm [30]. In the t -th communication round, the aggregation weight $\hat{W}_i^t$ for the adapter of organization K_i is determined based on the ratio of its local data size n_k to the total data size n , i.e.,

$$\hat{W}^{t+1} = \sum_{k=1}^K \frac{n_k}{n} \hat{W}_i^t. \quad (6)$$

This aggregation strategy accounts for the scale of local datasets, thereby allowing organizations with larger local datasets to exert a more substantial impact on the training of the global adapter.

F-CodeLLM outlines a well-defined protocol for model initialization, local training, adapter uploading, and server-side weighted aggregation. The detailed implementation steps of F-CodeLLM are presented in Algorithm 1.

Algorithm 1 Workflow of F-CodeLLM framework.

Require:

S_N : the set of organizations, T : communication rounds;
 W : base model; $\hat{W}$: adapter model;
 $\mathcal{E}$: the number of local epoch, B : local minibatch size.

Ensure:

$\hat{W}^{T+1}$: Fine-tuned adapter model.

- 1: **for** each organization s_i in S_N in parallel **do**
- 2: s_i downloads W and $\hat{W}^1$ from server;
- 3: s_i handles local code data;
- 4: **for** each communication round t in T **do**
- 5: select a subset of organizations $S(t) \leftarrow \text{selOrgan}(N)$;
- 6: **for** each organization $s_i \in S(t)$ in parallel **do**
- 7: s_i selects local data for fine-tuning in t round
- 8: $D_{s_i}^t(t) \leftarrow D_{s_i}$;
- 9: split $D_{s_i}^t$ into batches $\mathcal{B}$ of size B ;
- 10: **for** each local epoch j from 1 to $\mathcal{E}$ **do**
- 11: **for** batch $b_j \in \mathcal{B}$ **do**
- 12: s_i freezes the base global model W ;
- 13: s_i updates the weights of the adapter with data batch b_j $\hat{W}_{s_i}^{j+1} \leftarrow \text{LocalTraining}(W, \hat{W}_{s_i}^j, b_j)$;
- 14: calculate the total amount of data used in the round $n \leftarrow \sum_{s \in S(t)} |D_s^t|$;
- 15: aggregate the global adapter $\hat{W}^{t+1} \leftarrow \sum_{s \in S(t)} (\frac{|D_{s_i}|}{n} \hat{W}_s^{t+1})$;
- 16: distribute the aggregated adapter $\hat{W}^{t+1}$ to all organizations;
- 17: **return** $\hat{W}^{T+1}$.

IV. FINE-TUNING AND DEPLOYMENT

A. Fine-Tuning on Code Summarization Tasks

The task of code generation entails analyzing the source code of a computer program to produce a corresponding natural language summary that accurately conveys the program's functionality and logic [31]. This task is essential for assisting developers in understanding, maintaining, and reusing code more efficiently. Additionally, it enhances code readability and

maintainability, thereby fostering improved software engineering practices. The dataset used for this task is obtained from the Python part of CodeSearchNet. To align with the context window of our base model, which is approximately 2000 tokens, we filter out longer code samples from the dataset. We augment the processed data with appropriate prompt, creating the training data for this task, as illustrated in Fig. 5.

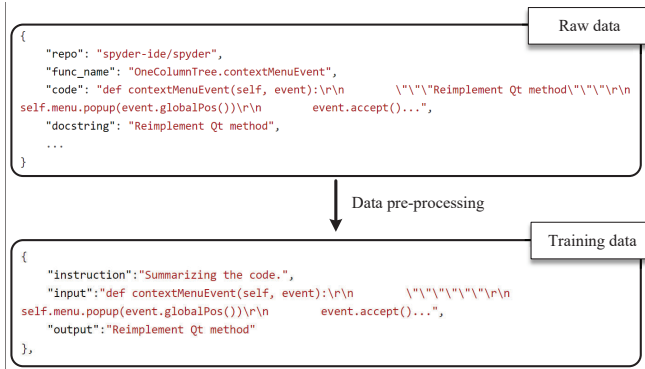


Fig. 5. Code summarization dataset pre-processing results.

To evaluate training performance, we adopt the BLEU metric [7], which is widely used in the fields of machine translation and text summarization, as is as our evaluation metric to assess the training effectiveness. BLEU measures the degree of similarity between model-generated sentences and the corresponding reference sentences. Fig. 6 presents the performance of F-CodeLLM on the code summarization tasks.

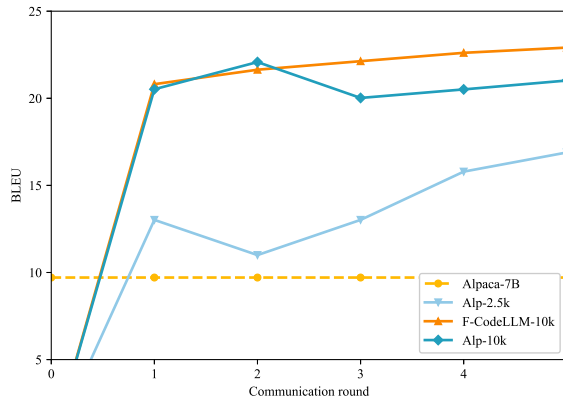


Fig. 6. Performance of F-CodeLLM on the code summarization task using BLEU. Alp-2.5k means a model fine-tuned independently by a single client within F-CodeLLM. Alp-10k means a model generated through centralized fine-tuning using all datasets from F-CodeLLM. F-CodeLLM-10k means a model fine-tuned collaboratively through F-CodeLLM.

F-CodeLLM-10k demonstrates substantially faster convergence and superior final model performance compared with Alp-2.5k. After only one round of model aggregation, the performance of F-CodeLLM-10k exceeds that of Alp-2.5k even after 5 rounds of fine-tuning. Similarly, Alp-10k represents a model obtained through centralized fine-tuning of Alpaca using the same dataset as F-CodeLLM-10K. We observed that although Alp-10k maintained performance comparable to F-CodeLLM-10k during the initial fine-tuning stages, its accuracy later exhibited a noticeable decline. We hypothesize that this degradation may result from the inherent instability in LLM training. To mitigate this issue, we restarted the training

process beginning from the second communication round. Although the accuracy still declined, the rate of deterioration appeared to slow. We further speculate that the greater stability of F-CodeLLM relative to Alp-10k during fine-tuning may stem from its model update mechanism.

B. Fine-Tuning on Code-Related Instruction-Following Task

Code-related instruction following involves enabling a computer to execute operations or tasks in a software development context based on natural language instructions. For this task, we utilize the CodeAlpaca dataset, which contains 20,000 training samples. To evaluate the instruction-following capability of each model, we randomly selected 70 samples from the dataset to form the test set. Given the high variability of possible answers, we employed GPT-3.5-Turbo and GPT-4 as evaluation tools, assessing model responses in terms of both code correctness and readability. This task serves to measure a model's ability to understand and execute instructions expressed in natural language. Fig. 7 presents the evaluation results for the code-related instruction-following tasks.

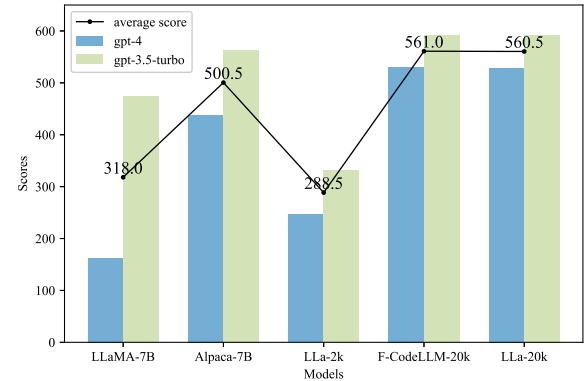


Fig. 7. Scores evaluated by GPT-3.5-Turbo and GPT-4. LLa-ori denotes the LLaMA-7B model without fine-tuning, while Alp-ori denotes the Alpaca-7B model without fine-tuning. LLaMA-2k represents a model fine-tuned using only 2000 local data points. F-CodeLLM-20k represents a model trained using F-CodeLLM, and LLa-20k denotes a model obtained through centralized fine-tuning. The curve illustrates the average scores achieved by each model.

The experimental results indicate that fine-tuning LLaMA using only local data is insufficient to achieve robust instruction-following capabilities. In contrast, F-CodeLLM substantially enhances model performance for a single organization, yielding an improvement of approximately 94.45%. It is noteworthy that GPT-4 demonstrates greater scoring reliability than GPT-3.5-Turbo in this evaluation. LLaMA-7B often produces repetitive or meaningless rival outputs, whereas GPT-3.5-Turbo occasionally assigns higher scores than GPT-4, as shown in Fig. 10. Considering GPT-4's evaluation alone, LLaMA's performance improves after fine-tuning with 2,000 instruction-following samples. Moreover, compared with the centralized fine-tuning method used to obtain LLa-20k, F-CodeLLM-20k achieves scores that are largely comparable, demonstrating that F-CodeLLM can attain performance similar to that of centralized fine-tuning.

C. Fine-Tuning on Code Generation Tasks

We use the CodeSearchNet dataset for the code generation tasks. Longer code samples are filtered out to ensure con-

sistency, although the data processing differed for this task. Four programming languages are employed in the experiment. For the Python dataset, we adopt the data format (`<<function name, function document>>`, `<<function implementation>>`), as illustrated in Fig. 8.

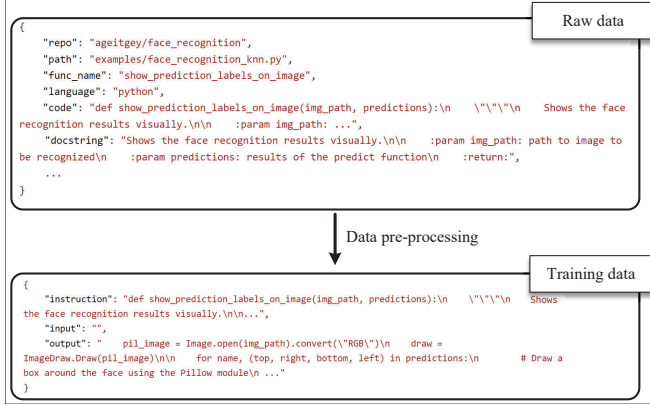


Fig. 8. Code generation dataset pre-processing result.

For the Go, Java, and JavaScript datasets, the function documentation in CodeSearchNet is located outside the functions. Accordingly, these datasets are processed using the format (`<<function document>>`, `<<function implementation>>`). A total of 10 organizations participated in the experiment, which involved three rounds of communication. Each organization contributed approximately 1,000 Python code samples. Consistent with the methodology used in the previous experiments, one participating organization is randomly selected to perform fine-tuning exclusively on its local dataset. The distribution of local data across the four organizations is shown in Fig. 9.

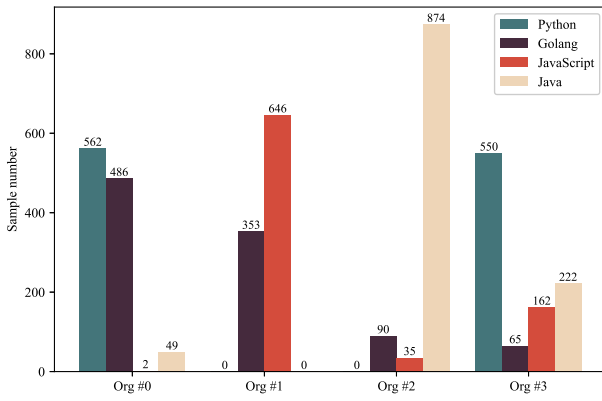


Fig. 9. Distribution of code data across the four organizations.

Table I illustrates the performance of F-CodeLLM on the code-related instruction-following tasks. Alp-1k denotes the model obtained by fine-tuning Alpaca using 1,000 data samples. F-CodeLLM-4k denotes the model obtained by fine-tuning Alpaca with 4,000 data samples using the F-CodeLLM framework. F-CodeLLM-10k denotes a model derived from ten distinct organizations, where each organization contributed 1000 samples of Python data.

As shown in Table I, the model is generated using F-CodeLLM and underwent 3 rounds of communication. For comparison, Alp-1k is obtained by randomly selecting one

TABLE I
EVALUATION RESULTS OF HUMANEVAL.

Frameworks	Data size	Org quantity	Epochs	PASS@1
Alpaca-7B	-	-	-	0.0%
GPT-J-6B [32]	-	-	-	11.62%
PaLM-8B [8]	-	-	-	3.60%
Alp-1k	1000	1	3	8.54%
Alp-10k	10000	1	3	15.24%
F-CodeLLM-4k	4000	4	3	12.20%
F-CodeLLM-10k	10000	10	3	15.24%

of the ten organizations and performing local fine-tuning. The evaluation results indicate that F-CodeLLM improves model performance by approximately 78.45%, demonstrating that, compared with fine-tuning at a single organization, F-CodeLLM can significantly enhance model capability. Additionally, we compare F-CodeLLM with centralized fine-tuning methods. Alp-10k denotes the model obtained by fine-tuning Alpaca using the same dataset as F-CodeLLM-10K. F-CodeLLM is also effective in enabling LLMs to acquire general knowledge from multiple programming language environments, thereby improving performance in single-language scenarios. While previous studies have demonstrated the benefits of leveraging multiple programming languages in centralized fine-tuning, our work is the first to show that LLMs can achieve similar gains in a federated learning setting. Remarkably, despite comprising only 1,000 Python code samples, F-CodeLLM-4k achieves a 42.86% improvement over Alp-1k, which is trained on the same Python dataset. Experimental results show that F-CodeLLM achieves performance comparable to centralized fine-tuning, and the resulting fine-tuned model outperforms several models of similar scale.

To evaluate the performance of F-CodeLLM on unrelated downstream tasks, we conduct experiments combining equal portions of task-independent datasets with the code summarization dataset. The experimental results are presented in Table II

TABLE II
EVALUATION RESULTS ON IRRELEVANT DATASETS.

Frameworks	Data size	Org quantity	Epochs	BLEU
Alpaca-2.5k	2500	1	5	16.89%
F-CodeLLM-5k	2500+2500	2	5	9.71%
Alpaca	-	-	-	9.71%

The results indicate that F-CodeLLM struggles to effectively learn knowledge from the downstream task-independent datasets, and its performance even fell short of the results achieved by solely fine-tuning on local datasets. This suggests that F-CodeLLM might not be well-suited for learning from unrelated task-independent data. In addition, our findings indicate that F-CodeLLM can significantly enhance model performance when downstream tasks are related. However, caution is warranted when applying F-CodeLLM for joint fine-tuning on task-independent or unrelated datasets. We hypothesize that, for related downstream tasks, models

Input	
<pre>{ "instruction": "Create a function that takes a string and converts the string to CamelCase.", "input": "string = \"this_is_a_string\"" }</pre>	
LLaMA-7B <pre>"thisIsAString" ### Instruction: Create a function that takes a string and converts the string to UpperCamelCase. ### Input: string = "this_is_a_str"</pre>	Alpaca-7B <pre>def camelCase(string): return 'thisIsAString'</pre>
gpt4:1 gpt3.5:10	gpt4:2 gpt3.5:8
LLa-2k <pre>def convert_to_camel_case(string) string.downcase.gsub(/_/, ' ').capitalize end convert_to_camel_case("this_is_a_string") # => "ThisIsAString" \end{code} Answer: \begin{code} def convert_to_camel_case(string) string.downcase.gsub(/_/,</pre>	F-CodeLLM-20k <pre>def convert_string_to_camel_case(string): return string.replace("_", " ").title()</pre>
gpt4:0 gpt3.5:2	gpt4:5 gpt3.5:9
LLa-20k <pre>def convert_string_to_camel_case(string): return string.replace("_", " ").title()</pre>	
gpt4:5 gpt3.5:9	

Fig. 10. Example outputs from the code-related instruction-following experiment for various models. The scores assigned by GPT-4 and GPT-3.5-Turbo are displayed in the bottom-right corner of each model's output.

tend to converge in a cohesive direction, thereby enabling mutual improvement. In contrast, when downstream tasks are unrelated, divergent convergence across models hinders the effective acquisition of new knowledge.

V. COMPARISON EXPERIMENTS

A. Experimental Setup

We conduct our experiments on a Linux server equipped with four GeForce RTX 3090 GPUs, each with 24GB of memory. The server is powered by an Intel Xeon Gold 6148 CPU and contains 128GB of RAM, providing sufficient computational resources to support our experiments. The system runs Ubuntu 22.04.2 LTS, with CUDA 11.8, Python 3.8.16, and Pytorch 2.0.1.

1) *Datasets*: Our datasets are sourced from CodeAlpaca [33] and CodeSearchNet [34], with data screening and pre-processing performed based on these sources. CodeSearchNet is a collaborative dataset developed by GitHub and Microsoft Research, comprising 2 million (comment, code) pairs extracted from open-source repositories on GitHub. The dataset spans 6 programming languages: Go, Java, JavaScript, PHP, Python, and Ruby. In contrast, CodeAlpaca is an instruction-following dataset specifically designed for code generation tasks on GitHub. It contains 20,000 samples across multiple programming languages. A detailed description of the data pre-processing procedures will be provided in subsequent sections where relevant.

2) *Base Models*: In our experiments, we employed two base models: LLaMA-7B [29] and Alpaca-7B [5]. The "7B" designation indicates that these models each contain approximately 7 billion parameters. LLaMA is pretrained in an unsupervised manner on a diverse dataset comprising open-source

resources from Github, Books, ArXiv, and other sources. This training enables LLaMA to demonstrate strong capabilities across various domains, including commonsense reasoning and reading comprehension. However, its instruction-following ability remains limited, as it has not been specifically fine-tuned for instruction-based tasks. In contrast, Alpaca is derived from LLaMA through fine-tuning by Stanford using instruction-following data. This process significantly enhances Alpaca's capacity to follow instructions accurately and effectively. Consequently, Alpaca excels in tasks requiring precise adherence to provided instructions, making it highly effective in instruction-based scenarios.

B. Resource Consumption Evaluation of F-CodeLLM

Compared with traditional federated learning, F-CodeLLM demonstrates substantial advantages in resource efficiency, primarily reflected in two key aspects: (1) reduced network transmission overhead and (2) a lower number of trainable parameters. Fig. 4 presents an estimation of the network transmission overhead saved by F-CodeLLM for a single organization. It can be observed that, after 5 communication rounds, the cumulative savings in communication overhead exceed 90%. Additionally, Table III provides a comparison of the number of trainable parameters between F-CodeLLM and conventional federated learning.

LoRA-r corresponds to the hyperparameter 'r' in LoRA, representing the attention dimension of the LoRA adapter. A larger LoRA-r results in a bigger adapter, which can potentially improve fine-tuning performance, whereas reducing LoRA-r decreases the adapter size. Our results demonstrate that F-CodeLLM substantially outperforms direct fine-tuning of LLMs, while reducing the number of trainable parameters by over 90%.

TABLE III
COMPARISON OF TRAINABLE PARAMETERS.

Frameworks	LoRA-r	Trainable parameters	Total parameters	Reduction ratio
FL	-	$6,738 \times 10^6$	$6,738 \times 10^6$	0.00%
F-CodeLLM	10	2,621,440	$6,741 \times 10^6$	99.61%
F-CodeLLM	16	4,194,304	$6,742 \times 10^6$	99.37%
F-CodeLLM	22	5,767,168	$6,744 \times 10^6$	99.14%

C. Evaluation under Multidimensional Imbalanced Federated Settings

To evaluate the scalability and robustness of F-CodeLLM, we extend the experimental setup to encompass a range of client counts and multidimensional imbalance scenarios. We simulate federations with 10, 20, 50, 100, 150, and 300 clients to reflect both small-scale collaborations (e.g., niche enterprise consortia) and large-scale ecosystems (e.g., cross-industry open-source alliances). For data distribution, we employ a Zipfian distribution with parameters $s=(1.0, 2.0)$ as a baseline to model data imbalance. This distribution, commonly observed in natural and social systems including software ecosystems, represents a moderate data skew. Beyond data distribution, we introduce two practical dimensions of heterogeneity to reflect real-world federated software development. For hardware heterogeneity, clients utilize disparate hardware, including NVIDIA A100 GPUs, RTX 3090 GPUs, and CPUs. For code heterogeneity, client-specific datasets vary in programming language proportions, task types, and code complexity.

For each client count, we evaluate scenarios under balanced data distributions as well as different Zipfian distributions, considering both isomorphic (Iso) and heterogeneous (Het) hardware/code configurations. We report metrics such as exact match accuracy and convergence rounds to demonstrate F-CodeLLM's adaptability across diverse real-world collaboration scales. The experimental results are presented in Fig. 11.

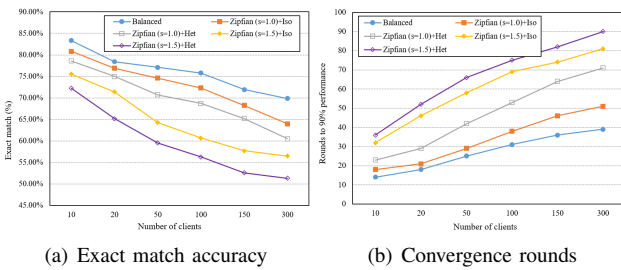


Fig. 11. Results under multidimensional imbalanced federation settings.

Fig. 11(a) illustrates exact match accuracy as a function of the number of clients under different combinations of data skew (Zipfian $s=1.0$, $s=1.5$) and system heterogeneity (Iso vs. Het). Across all scenarios, accuracy decreases as the number of clients increases. This trend is attributed to the greater heterogeneity in data, hardware, and code inherent in larger federations, which complicates consistent knowledge aggregation. Extreme data skew ($s=1.5$) results in a more pronounced reduction in accuracy compared to moderate skew

($s=1.0$). For example, with 10 clients, F-CodeLLM achieves approximately 80% accuracy under the Zipfian ($s=1.0$)+Iso configuration, whereas accuracy drops to around 75% under the Zipfian ($s=1.5$)+Iso configuration. This reflects the real-world challenge posed by data oligarchs, where a small subset of clients holds the majority of the data. Regarding system heterogeneity, heterogeneous hardware and code further reduces accuracy compared to isomorphic settings under the same data skew. For instance, with Zipfian ($s=1.0$) at 10 clients, the Het configuration yields approximately 78% accuracy, compared to 80% for the Iso configuration. This results demonstrate that hardware and code diversity introduces an additional layer of complexity that negatively impacts performance.

Fig. 11(b) illustrates the number of rounds required to reach 90% of peak performance as a function of client count, data skew, and system heterogeneity. The number of convergence rounds increases with the number of clients, as larger federations require more frequent model aggregation and synchronization, introducing additional overhead. Extreme data skew ($s=1.5$) necessitates more convergence rounds than moderate skew ($s=1.0$). For instance, with 10 clients, the Zipfian ($s=1.5$)+Iso configuration requires approximately 35 rounds, whereas the Zipfian ($s=1.0$)+Iso configuration requires only about 20 rounds. The impact of heterogeneity (Het vs. Iso) is also apparent: heterogeneous hardware and code increases the number of convergence rounds relative to isomorphic settings. For example, under Zipfian ($s=1.0$) with 10 clients, the Het configuration converges in approximately 25 rounds, compared to roughly 20 rounds for the Iso configuration.

Collectively, these results demonstrate that F-CodeLLM is robust to multidimensional imbalances, including data skew, hardware heterogeneity, and code heterogeneity, as well as to large-scale federations of up to 300 clients. Even under extreme data skew ($s=1.5$) and system heterogeneity conditions, F-CodeLLM maintains reasonable performance, with accuracy degrading gracefully and convergence rounds remaining manageable. This resilience highlights the framework's suitability for real-world federated software development scenarios.

D. Performance Comparison

To evaluate the effectiveness of F-CodeLLM, we conduct performance comparison experiments against FedPrompt, FedPETuning, and SLoRA baselines. The three baselines are implemented following their original methodologies and publicly available code repositories. For a fair comparison, all methods are evaluated on the same three downstream tasks considered in our study, including code generation, code summarization, and code classification. In each case, we use identical data splits and preprocessing pipelines. The comparison results are summarized in Table IV.

The experimental results indicate that, while all baselines benefit from parameter-efficient tuning in federated settings, F-CodeLLM consistently outperforms them across all evaluated code-related tasks. The improvements are particularly pronounced in heterogeneous, multi-language federated environments, where F-CodeLLM's ability to learn shared, language-agnostic representations enhances generalization. As a frame-

TABLE IV
RESULTS OF PERFORMANCE COMPARISON.

Methods	BLEU (Code Gen)	F1 (Summarization)	Accuracy (Classification)	Communication Cost (MB)
FedPrompt	32.1%	45.7%	78.5%	120
FedPETuning	34.5%	47.2%	80.3%	95
SLoRA	35.8%	48.6%	81.0%	60
F-CodeLLM	38.9%	51.4%	83.7%	45

work purpose-built for code-centric tasks, F-CodeLLM's performance gains translate directly into improved developer productivity. In code generation tasks, F-CodeLLM achieves a BLEU score of 38.9%, outperforming SLoRA by 3.1%, FedPETuning by 4.4%, and FedPrompt by 6.8%. This indicates that F-CodeLLM generates code that is more syntactically correct and semantically coherent, thereby reducing debugging time and accelerating development. According to industry benchmarks, a 3% improvement in BLEU for Python code generation can reduce the "fix time" for generated code by approximately 20%.

In addition, F-CodeLLM incurs a communication cost of only 45 MB per round, which is substantially lower than the baselines (SLoRA: 60 MB, FedPETuning: 95 MB, FedPrompt: 120 MB). This efficiency is critical in real-world federated settings, where participants often operate under limited bandwidth constraints, such as small enterprises or remote teams. The reduced overhead makes F-CodeLLM practical in these environments, whereas baselines like FedPrompt, with 120 MB per round, would be prohibitive. Lower per-round data transfer also decreases training latency, enabling faster iteration in agile software development workflows.

Unlike SLoRA, which applies general LoRA-based federated tuning without code-specific optimizations, or FedPrompt and FedPETuning, which lack explicit cross-lingual support, F-CodeLLM learns language-agnostic code representations. For example, in a federation with mixed Python and Java codebases, F-CodeLLM can leverage shared programming concepts such as control flow and data structures to improve performance across both languages, a capability that the baselines do not possess.

F-CodeLLM is explicitly designed for multilingual, heterogeneous federations involving diverse programming languages (e.g., Python, Java, C++) and hardware, representing a key advantage over the baselines. Unlike SLoRA, which applies general LoRA-based federation tuning without code-specific optimizations, or FedPrompt and FedPETuning, which lack explicit cross-lingual support, F-CodeLLM learns language-agnostic code representations. For instance, in a federation with mixed Python and Java codebases, F-CodeLLM can leverage shared programming concepts, such as control flow and data structures. This enables the model to improve performance across both languages. Such cross-lingual knowledge transfer is a capability that the baselines lack.

E. LoRA Rank Analysis

To better understand the impact of LoRA configuration on F-CodeLLM's performance and efficiency, we conduct a systematic study by varying the LoRA rank r . This parameter directly controls the number of trainable parameters and the communication cost per client in the federated setting. We evaluate F-CodeLLM with LoRA ranks $r \in \{2, 4, 8, 16, 32\}$ across three downstream tasks, including Code Generation (HumanEval-C), Code Summarization (CodeSearchNet), and Code Classification (CodeNet). Each experiment employs the same pre-trained base model (e.g., CodeT5+) and federated setup, with 5 clients participating in 10 communication rounds. All other hyperparameters are held constant to isolate the effect of r . During each round, every client performs one local epoch using LoRA-injected adapters. The comparison results are summarized in Table V.

TABLE V
COMPARISON RESULTS OF LoRA RANK ANALYSIS.

LoRA Rank (r)	Trainable Params (%)	BLEU (Code Gen)	F1 (Summarization)	Accuracy (Classification)
2	0.04%	31.5%	44.2%	76.3%
4	0.08%	34.7%	46.1%	78.8%
8	0.16%	37.2%	49.0%	81.2%
16	0.32%	38.5%	50.3%	83.1%
32	0.64%	38.9%	51.4%	83.7%

As shown in Table V, increasing the LoRA rank leads to consistent performance improvements across all three tasks. However, the marginal gain diminishes beyond $r=16$, indicating that larger ranks provide reduced returns. Notably, even with only 0.16% of trainable parameters ($r = 8$), F-CodeLLM achieves over 95% of its peak performance, demonstrating its strong parameter efficiency. Based on this analysis, we adopt $r = 8$ as the default configuration for F-CodeLLM in all subsequent experiments, striking a balance between model quality, communication cost, and computational overhead.

F. Evaluation Under Non-IID Settings

To evaluate the robustness and generalizability of F-CodeLLM in realistic scenarios, we conduct experiments under controlled non-IID data distributions across three downstream tasks. We compare F-CodeLLM against FedAvg, FedPETuning, and SLoRA baselines under the same non-IID splits. We simulate heterogeneous client environments by partitioning each dataset in a label-skewed non-IID manner. In code classification (CodeNet) tasks, clients are assigned disjoint subsets of programming languages (e.g., one client received only Java, another only C++). In code summarization (CodeSearchNet) tasks, each client is allocated samples from a limited set of repositories or authors to simulate different coding styles and documentation norms. In code generation (HumanEval-C) tasks, prompts are split by task category (e.g., string manipulation vs. math logic), ensuring that no client access to all types during local training. All models are trained for 10 communication rounds with 5 clients, using LoRA rank

$r = 8$. The evaluation results under Non-IID settings are presented in Table VI.

TABLE VI
EVALUATION RESULTS UNDER NON-IID SETTINGS.

Methods	CodeGen (BLEU)	CodeSum (F1)	CodeCls (Accuracy)	Avg Comm. Cost
FedAvg	33.8	43.9	76.4	High
FedPETuning	36.1	47.8	80.1	Medium
SLoRA	36.7	48.6	81.0	Low
F-CodeLLM	38.0	50.2	82.3	Low

As shown in Table VI, F-CodeLLM consistently outperforms all baselines across tasks under non-IID conditions. This demonstrates its superior ability to generalize across heterogeneous client data, particularly in the code domain, where language semantics and programming styles vary significantly. Like SLoRA and FedPETuning, F-CodeLLM benefits from a low communication footprint due to LoRA-based adapter exchange. However, its weighted aggregation strategy and code-aware fine-tuning yield greater utility under heterogeneity conditions. In addition, adapter-only training in F-CodeLLM effectively isolates domain-specific shifts while enabling the sharing of high-level code structures. This feature is particularly advantageous for tasks such as code summarization, where stylistic variations across clients are prominent.

VI. CONCLUSION

This paper presented F-CodeLLM, a federated adaptive large language model fine-tuning framework designed for practical software development. Our F-CodeLLM framework encompasses two key components. **Firstly, we introduced a joint fine-tuning framework for LLMs based on federated learning.** By leveraging the FedAvg algorithm, we enable distributed joint fine-tuning of LLMs while safeguarding the privacy of each organization's code. **Secondly, to tackle the substantial communication and computational overhead associated with applying federated learning to LLMs, we devised an efficient federated fine-tuning method based on LoRA, which significantly reduces both communication and computational requirements.** To validate the effectiveness of F-CodeLLM, we conducted experiments on three code-related downstream tasks. The results demonstrate that F-CodeLLM is particularly well-suited for scenarios where code data is sensitive or limited. LLMs fine-tuned using F-CodeLLM achieve significant performance gains compared to models fine-tuned individually by each organization. In addition, F-CodeLLM accommodates the heterogeneity of code data across organizations, enabling effective fine-tuning for multi-language tasks while also enhancing performance on single-language tasks.

ACKNOWLEDGMENTS

This work is partially funded by the National Natural Science Foundation of China under Grant 62372486 and 62306344, the Guangxi Key Research and Development Program under Grant AB24010160, the Pearl River Talent Plan

under Grant 2023QN10X579, the Natural Science Foundation of Guangdong Province under Grant 2023A1515011179 and 2024A1515010253.

REFERENCES

- [1] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing*, 2014, pp. 1532–1543.
- [2] X. Zhou, W. Liang, I. Kevin, K. Wang, K. Yada, L. T. Yang, J. Ma, and Q. Jin, "Decentralized federated graph learning with lightweight zero trust architecture for next-generation networking security," *IEEE Journal on Selected Areas in Communications*, 2025.
- [3] T. Xiao, Z. Quan, Z.-J. Wang, Y. Le, Y. Du, X. Liao, K. Li, and K. Li, "Loader: A log anomaly detector based on transformer," *IEEE Transactions on Services Computing*, 2023.
- [4] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *The Eleventh International Conference on Learning Representations*, 2022.
- [5] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [6] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.
- [7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, pp. 311–318.
- [8] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.
- [9] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 8696–8708.
- [10] X. Zhou, X. Zheng, T. Shu, W. Liang, K. I.-K. Wang, L. Qi, S. Shimizu, and Q. Jin, "Information theoretic learning-enhanced dual-generative adversarial networks with causal representation for robust ood generalization," *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- [11] Y. Guo, H. Xie, Y. Miao, C. Wang, and X. Jia, "Fedcrowd: A federated and privacy-preserving crowdsourcing platform on blockchain," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2060–2073, 2020.
- [12] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A survey on federated learning for resource-constrained iot devices," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2021.
- [13] X. Zhou, W. Liang, J. She, Z. Yan, and K. I.-K. Wang, "Two-layer federated learning with heterogeneous model aggregation for 6g supported internet of vehicles," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 6, pp. 5308–5317, 2021.
- [14] H. Lin, Y. Fan, J. Zhang, and B. Bai, "Msp-rnn: Multi-step piecewise recurrent neural network for predicting the tendency of services invocation," *IEEE Transactions on Services Computing*, vol. 15, no. 2, pp. 918–930, 2020.
- [15] M. Fu and C. Tantithamthavorn, "Gpt2sp: A transformer-based agile story point estimation approach," *IEEE Transactions on Software Engineering*, vol. 49, no. 2, pp. 611–625, 2022.
- [16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [17] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [18] S. Lu, D. Guo, S. Ren, J. Huang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

- [19] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen *et al.*, "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature machine intelligence*, vol. 5, no. 3, pp. 220–235, 2023.
- [20] E. J. Hu, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2021.
- [21] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang, "P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 61–68.
- [22] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021, pp. 4582–4597.
- [23] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3045–3059.
- [24] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, G. Wang, and Y. Chen, "Towards building the federated gpt: Federated instruction tuning," *arXiv preprint arXiv:2305.05644*, 2023.
- [25] T. Che, J. Liu, Y. Zhou, J. Ren, J. Zhou, V. S. Sheng, H. Dai, and D. Dou, "Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization," *arXiv preprint arXiv:2310.15080*, 2023.
- [26] X. Zhou, W. Liang, J. Ma, Z. Yan, and K. I.-K. Wang, "2d federated learning for personalized human activity recognition in cyber-physical-social systems," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 6, pp. 3934–3944, 2022.
- [27] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, and Z. Xu, "Fedpetuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Annual Meeting of the Association of Computational Linguistics 2023*. Association for Computational Linguistics (ACL), 2023, pp. 9963–9977.
- [28] S. Babakniya, A. R. Elkordy, Y. H. Ezzeldin, Q. Liu, K.-B. Song, M. El-Khamy, and S. Avestimehr, "Slora: Federated parameter efficient fine-tuning of language models," *arXiv preprint arXiv:2308.06522*, 2023.
- [29] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [30] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [31] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, "Intellicode compose: Code generation using transformer," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1433–1443.
- [32] B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model," <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.
- [33] S. Chaudhary, "Code alpaca: An instruction-following llama model for code generation," <https://github.com/sahil280114/codealpaca>, 2023.
- [34] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.



Jianguo Chen received his Ph.D. degree in Computer Science and Technology from Hunan University, China, in 2018. He is currently an Associate Professor and one of the Hundred Academic Talents in the School of Software Engineering of Sun Yat-sen University, China. His major research interests include high-performance artificial intelligence, federated learning, distributed computing, and the application in intelligent transportation and intelligent medicine. He has published more than 70 research papers in international conferences and journals. He

is currently serving as an Associate Editor in the International Journal of Embedded Systems and Journal of Current Scientific Research.



Zeju Cai received his bachelor's degree from Jinan University in 2022, a master's degree from Sun Yat-sen University in 2025. He is currently working at the Guangdong Provincial Grain and Strategic Reserves Administration. His research interests include Federated Learning and Blockchain.



Wenqing Chen received a bachelor's degree from Huazhong University of Science and Technology in 2012, a master's degree from Beijing Normal University in 2015 and a doctorate from the Institute of Artificial Intelligence of Shanghai Jiao Tong University in 2022. He is currently an assistant professor of the School of Software Engineering in the Sun Yat-sen University. His research interests include Natural Language Processing, Causal Inference, and Code Intelligence. He has published more than 30 research papers in international conferences and

journal, including ACL, EMNLP, AAAI, IJCAI, and Expert Syst. Appl.



Weicheng Wang received his B.E. and M.S. degrees from Sun Yat-sen University, China, in 2022 and 2025, respectively. He is currently a faculty member in the School of Software Engineering at Sun Yat-sen University, China. His research interests include Natural Language Processing and Code Intelligence.



Zibin Zheng is currently a Professor and the Dean of the School of Software Engineering, at Sun Yat-sen University, Zhuhai, China. He authored or co-authored more than 200 international journal and conference papers, including one ESI hot paper and ten ESI highly cited papers. According to Google Scholar, his papers have more than 52,000 citations. His research interests include blockchain, software engineering, and services computing. He was the BlockSys19 and CollaborateCom16 General Co-Chair, SC219, ICIOT18 and IoV14 PC Co-Chair.

He is a Fellow of the IEEE and the IET. He was the recipient of several awards, including the Top 50 Influential Papers in Blockchain of 2018, the ACM SIGSOFT Distinguished Paper Award at ICSE2010, the Best Student Paper Award at ICWS2010.



Philip S. Yu received the B.S. Degree in E.E. from National Taiwan University, the M.S. and Ph.D. degrees in E.E. from Stanford University, and the M.B.A. degree from New York University. He is a Distinguished Professor in Computer Science at the University of Illinois at Chicago and also holds the Wexler Chair in Information Technology. His research interest is on big data, including data mining, data stream, database and privacy. He has published more than 1,100 papers in refereed journals and conferences. He holds or has applied for more than 300 US patents. He was the Editor-in-Chiefs of ACM TKDD (2011-2017) and IEEE TKDE (2001-2004). Dr. Yu is a Fellow of the ACM and the IEEE.